Estudo: client remoto (instalador + agent leve CDP via WebSocket)

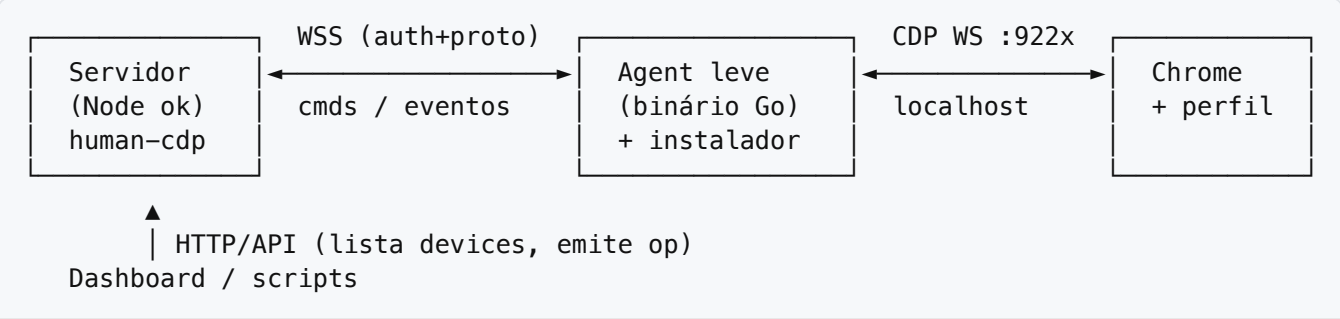
Card: 3Glk2RLz · **Data:** 2026-11-03

Contexto local: `human-cdp` + `start-*.sh` no lab (`pocs/chrome-cdp/`)

1. Problema → proposta

Hoje o Chrome com `--remote-debugging-port` e a lib `human-cdp` vivem na mesma máquina do operador. Para frota em PCs de terceiros, a pessoa só deve instalar um pacote leve; o **servidor** cadastra o device e emite operações CDP — sem Node/npm no client e **sem expor a porta CDP na internet**.

Modelo escolhido: agent **outbound** (client diala WSS para o servidor). Funciona atrás de NAT/firewall; o servidor nunca abre porta no PC da pessoa.



Camada humana: fica no **servidor** (reusa `human-cdp`). O agent é um **pipe CDP** + lifecycle do Chrome — não reimplementa trajetória no client na v1.

2. Instalador (Windows + Linux + macOS)

Decisão: três instaladores (um por OS), **mesmo código Go** do agent. Não um instalador universal — cada OS tem empacotamento e expectativa de UX diferentes; forçar um único pacote piora a experiência sem ganho real.

OS	Artefato de distribuição	Por quê separado
Windows	`sr-agent-setup.exe` (Inno Setup / WiX) ou ZIP + `sr-agent.exe` + serviço	`.exe` /serviço Windows, paths `%LOCALAPPDATA%` , Chrome em `Program Files`
macOS	`.pkg` ou app `.app` (notarização depois) + LaunchAgent	Gatekeeper, `~/Library/Application Support` , launchd
Linux	`.deb` / `.rpm` **ou** tarball `tar.gz` + unit systemd	distros variam; tarball+systemd cobre a maioria na POC

O que é único entre OSes: o binário `sr-agent` (Go, `GOOS` / `GOARCH` distintos) + o **protocolo** e a pasta de perfil. O instalador só: coloca o binário, registra serviço/daemon, pede o token, sobe o agent.

Item	Decisão v1
OS alvo	**Windows, Linux e macOS** (amd64 + arm64 onde fizer sentido)
Código	**Um** projeto Go; CI gera 3× binários + 3× pacotes
Chrome	Pré-instalado **ou** detector + link oficial; agent lança com flags CDP
Perfil	Windows `%LOCALAPPDATA%/sr-agent/profiles/<id>/` · macOS `~/Library/Application Support/sr-agent/profiles/<id>/` · Linux `~/.local/share/sr-agent/profiles/<id>/`
Permissões	Rede egress HTTPS/WSS; sem Accessibility/Screen Recording (só CDP)
UX	"colar token" → serviço sobe → device aparece no servidor
Update	HTTP check + replace binário (assinatura); sem auto-update forçado na POC

Flags Chrome alinhadas ao lab: `--remote-debugging-port=<local>` `--user-data-dir=<perfil>` `--remote-allow-origins=*` (só localhost). Porta **só 127.0.0.1**.

3. Runtime do agent — comparação e escolha

Critério	Go	Rust	Python embutido	Node
Binário único	sim (~8–15 MB)	sim (~2–8 MB)	PyInstaller ~30–50 MB+	precisa runtime / pkg pesado
WebSocket	std / gorilla / nhooyr	tokio-tungstenite	websockets	nativo
CDP	`chromedp` + chromedp-proxy	crates ad-hoc	pychrome	já temos no lab
Cross-compile 3 OS	excelente (`GOOS/GOARCH`)	bom, mais atrito (linkers, targets)	frágil	médio
Manutenção (time JS/TS)	curva baixa	curva alta	média	alta no client (evitar)
Tamanho / RSS	bom	**melhor** (ganho típico ~2–8 MB)	pior	pior no client

O que é o "thin pipe" (e por que o tamanho do Rust quase não importa)

O agent **não** é um browser automation framework no PC da pessoa. Ele faz só isto:

1. Mantém **uma** conexão WSS outbound com o servidor (auth, heartbeat, reconexão).

2. Mantém **uma** conexão WS CDP com o Chrome em `127.0.0.1`.

3. Traduz envelopes do protocolo (`cdp.send` → JSON-RPC CDP; eventos CDP → `cdp.event`).

4. Cuida do **lifecycle** do Chrome (spawn/attach, porta, perfil).

Não há no client v1: trajetória de mouse, OCR, fila de jobs complexa, UI, motor de regras. Quase todo o trabalho pesado e a "inteligência" ficam no **servidor** (`human-cdp`). O agent é um **túnel**

autenticado + gerenciador mínimo do Chrome.

Consequência para o critério de tamanho:

- O download que a pessoa sente é **instalador + Chrome** (Chrome sozinho são **centenas de MB**). A diferença **Go ~10 MB vs Rust ~4 MB** é ruído (<2% do pacote típico).
- Em runtime, o processo que domina RAM/CPU é o **Chrome**, não o agent. Economizar 2–8 MB de RSS no agent não muda capacidade da máquina nem densidade de frota de forma mensurável.
- O thin pipe **aumenta** o valor de: (a) entregar rápido nos 3 OS, (b) CDP estável (`chromedp`), (c) time conseguir ler/corrigir o agent. **Diminui** o valor de micro-otimizar o binário — não há loop quente nem footprint crítico no agent.

Por que **não** Rust agora (mesmo sendo menor)

Rust ganha em tamanho e RSS. Isso **seria** diferencial se o agent fosse:

- embarcado / IoT com orçamento rígido de MB, **ou**
- um runtime gordo no client (replay, ML, muitos workers), **ou**
- o gargalo de distribuição (milhões de updates diários onde cada MB conta).

Nenhum desses é o caso. O agent é thin pipe + Chrome local; o custo real de escolher Rust seria:

- curva e hiring no time (hoje JS/TS + Go ok);
- menos peças prontas para CDP (vs `chromedp` / `chromedp-proxy`);
- cross-compile e packaging Windows/macOS/Linux com mais atrito para a mesma superfície de features.

Conclusão: o ganho de 2–8 MB do Rust **não justifica** o custo neste desenho. Go é a melhor solução **de fato** para este produto: um código, três binários/instaladores, CDP maduro, entrega mais rápida. Reavaliar Rust só se o agent deixar de ser thin pipe (lógica pesada no client) ou se houver requisito explícito de footprint.

Recomendação: **Go**

- Binário estático + cross-compile nativo para **Windows / Linux / macOS** (amd64/arm64).
- Ecossistema CDP maduro (`chromedp`, proxy de referência).
- Thin pipe → prioriza velocidade de entrega e manutenção, não o último MB.
- Card pede **não-Node** no client; Python embutido e Node violam runtime mínimo.

4. Controle CDP local (no agent)

Responsabilidades do agent:

1. Garantir Chrome up (spawn ou attach se porta já em LISTEN + mesmo perfil).

2. Descobrir `webSocketDebuggerUrl` via `http://127.0.0.1:PORT/json/version` e `/json/list`.

3. Manter 1 WS CDP local; multiplexar pedidos do servidor.

4. Encaminhar **JSON-RPC CDP** bidirecional (commands + events).

5. Telemetria: Chrome version, target count, RSS, uptime, lastError.

Não no agent v1: trajetória humana, OCR, `Page.bringToFront` / activate SO (mesmas regras do lab).

Referência útil: `chromedp-proxy` (proxy WS CDP + `/json`) — padrão de bridge, não produto final (ele *expõe* porta; nosso agent *sai* por WSS).

5. Canal WebSocket ↔ servidor

5.1 Transporte e auth

- URL: `wss://<host>/agent/v1?device_id=<id>`
- Auth: header `Authorization: Bearer <token>` **ou** 1ª mensagem `hello` com token (preferir header).
- Token = enrollment one-time ou device credential rotativo.
- TLS obrigatório fora do lab; lab pode `ws://127.0.0.1`.

5.2 Handshake / cadastro

```
// client → server
{ "type": "hello", "proto": 1, "token": "...", "meta": {
  "hostname": "macbook-ana",
  "os": "darwin", "arch": "arm64",
  "agentVersion": "0.1.0",
  "chrome": { "version": "131.0.x", "port": 9222, "profile": "default" },
  "caps": ["cdp.raw", "screenshot"]
}}

// server → client
{ "type": "welcome", "deviceId": "dev_…", "heartbeatSec": 20 }
```

Cadastro persiste: `deviceId`, meta, `status` (`online` | `offline` | `busy` | `error`), `lastSeen`.

5.3 Heartbeat / reconexão

- Client envia `{ "type": "ping", "ts": … }` a cada `heartbeatSec`; server responde `pong`.
- Drop → exponential backoff (1s…30s) + re-`hello` com mesmo `deviceId`/credencial.
- Server marca `offline` após 2–3 heartbeats perdidos.

5.4 Protocolo de operação (esboço)

Envelope comum:

```
{ "v": 1, "id": "msg_…", "type": "<kind>", "payload": { } }
```

type	Direção	Payload
`cdp.send`	S→C	`{ "method", "params", "sessionId?" }` — JSON-RPC CDP
`cdp.result`	C→S	`{ "id", "result" | "error" }` correlacionado ao `id` do send
`cdp.event`	C→S	`{ "method", "params", "sessionId?" }` (ex. `Target.*`, `Page.*`)

`chrome.ensure`	S→C	`{ "port?", "profile?", "url?" }` — sobe/reusa Chrome
`chrome.status`	C→S	estado local
`screenshot`	S→C	atalho → `Page.captureScreenshot` (opcional na POC)
`error`	C→S	`{ "code", "message" }`

Correlação: cada `cdp.send` usa `id`; agent ecoa no `cdp.result`.

Fluxo operação remota:

```
API/UI → server escolhe device online
      → WSS cdp.send (Input.dispatchMouseEvent / Page.navigate / …)
      → agent → CDP localhost
      → cdp.result (+ cdp.event assíncronos)
      → human-cdp no server orquestra sequência (click = vários sends)
```

Assim `createCdpSession` do lab pode ganhar um transport "remote" que fala o proto em vez de `WebSocket(wsUrl)` local.

6. Operação pelo servidor

Peça	Papel
Registry de devices	CRUD mínimo (id, meta, status, lastSeen) — espelha ideia do CRUD CDPs
Hub WSS	conexões agent + roteamento por `deviceId`
API	`GET /devices`, `POST /devices/:id/cdp` (body = method/params)
Adapter	`human-cdp` com transport remoto
UI frota	**fora** — card A1h6DvaX

POC pode ser só CLI/API sem dashboard.

7. Riscos

Risco	Mitigação
Token vazado = controle do Chrome	token por device, rotação, escopo mínimo, revoke
CDP na LAN/internet	bind **127.0.0.1** apenas; nunca abrir 9222 externo
Superfície do agent	binário assinado; sem shell remoto genérico na v1
Chrome ausente / porta ocupada	`chrome.ensure` + erros claros; reusar se LISTEN + perfil ok

Update malicioso	HTTPS + checksum/assinatura
Foco / janela	mesma política lab: só `Emulation.setFocusEmulationEnabled`; sem activate SO
Firewall/NAT	outbound WSS resolve; proxy corporativo HTTP CONNECT a validar
Event flood CDP	filtrar domains; backpressure no hub
Multi-aba / sessionId	passar `sessionId` no envelope (como `human-cdp` já faz)

8. Fora de escopo / próximo passo (POC)

Fora (este card): instalador produção, multi-tenant, dashboard frota, agent com lógica humana embarcada.

POC sugerida (próximo card):

1. Binário Go mínimo: sobe/conecta Chrome local + WSS echo ao server Node de lab.

2. Mensagens: `hello` / `ping` / `cdp.send` / `cdp.result` / `cdp.event`.

3. Server: lista 1 device + `Page.navigate` + `Page.captureScreenshot` round-trip.

4. Opcional: plugar transport remoto no `createCdpSession` e rodar 1 teste `human-cdp` (click) contra o agent.

5. Critério de aceite: máquina "remota" (outro user-data-dir) operada só via WSS, sem Node no client.

9. Decisões resumidas

Tema	Decisão
Topologia	Agent outbound WSS; CDP só localhost
Runtime client	**Go** (1 codebase → binários por OS/arch)
Go vs Rust	Go: thin pipe + CDP maduro + 3 OS; ganho 2–8 MB do Rust não justifica
Lógica humana	Servidor (`human-cdp`); agent = pipe + Chrome lifecycle
Protocolo	JSON envelopes `hello` / `cdp.*` / heartbeat
Instalador	**3 pacotes** (Win/Linux/macOS), mesmo agent Go; token + perfil + Chrome detect/launch
Próximo	Card POC agent Go + hub WSS mínimo